Faculty of Engineering & Technology

Electrical & Computer Engineering Department

Advanced Digital – ENCS3310

Project Report – Comparator Designed

Prepared by:

Name: Yasmin Al Shawawrh. **Number:** 1220848.

Instructor: Dr. Abdellatif Abu-Issa.

Section: 4.

Date: 26/12/2024.

2024-2025

Abstract (Brief introduction)

This project focuses on designing a 6-bit comparator that can compare both signed and unsigned numbers using basic logic gates like AND, OR, and NOR. The comparator has three outputs: Equal, Greater, and Smaller, which indicate the relationship between two input numbers A and B, based on a selection bit (S) that determines whether the comparison is signed or unsigned. The design is implemented using structural Verilog with logic gates and clocked registers to ensure synchronous operation. Additionally, a behavioral Verilog model of the comparator is created to compare the results with the structural design. A testbench is written to validate both models, ensuring the system works correctly under all conditions and identifying any discrepancies for verification.

Table of Contents

Abstract (Brief introduction) I

Table of figures III

Brief theoretical overview 1

- **Comparator in digital system** 1
- **Signed vs Unsigned number** 1
- **Verilog Modeling Approaches** 1
- **Comparator Design Consideration** 1
- **Simulation and Testing in Verilog** 2

Design Philosophy 3

- **Structural Comparator** 3
- **Behavioral Comparator** 5
- **Testbench** 6

Simulation Results 9

Conclusion and Future works 12

Table of figures

FIGURE 1: STRUCTURAL COMPARATOR CODE..... 3
FIGURE 2: BEHAVIORAL COMPARATOR CODE..... 5
FIGURE 3: TESTBENCH CODE FOR ALL CASES. 6
FIGURE 4: TESTBENCH CODE FOR SOME CASES..... 7
FIGURE 5: SOME RESULTS FOR TESTBENCH1. 9
FIGURE 6: RESULTS FOR TESTBENCH2. 9
FIGURE 7: CODE WITH ERROR IN LINE 44 USE OR GATE INSTEAD OF AND GATE. 9
FIGURE 8: WRONG OUTPUT. 10
FIGURE 9:6-BIT COMPARATOR..... 10
FIGURE 10: CODE BLOCK DIAGRAM. 11

Brief theoretical overview

- **Comparator in digital system**

  A comparator is a digital circuit used to compare two binary numbers, A and B, and determine their relationship—whether A is equal to, greater than, or less than B. Comparators are essential in arithmetic and control systems, as they enable decision-making based on numerical values. In digital design, comparators can be signed or unsigned, depending on whether the numbers are interpreted as integers with or without sign representation. The key principles include bitwise comparison and handling of sign bits in signed operations, often utilizing two's complement for negative numbers. These features make comparators versatile components in complex digital systems.

- **Signed vs Unsigned number**

  Unsigned comparisons treat all bits of a number as positive values, meaning there are no negative numbers. The comparator directly compares the binary values without considering a sign bit. In contrast, signed comparisons use the most significant bit (MSB) to indicate whether a number is positive or negative. For unsigned numbers, the comparison is straightforward, but for signed numbers, additional checks are needed to account for the sign, especially when one number is positive and the other is negative.

- **Verilog Modeling Approaches**

  Verilog has three main modeling approaches: structural, dataflow, and behavioral. Structural modeling represents hardware using interconnected components, like building a circuit. Dataflow focuses on how data moves and is processed using logical and arithmetic expressions. Behavioral modeling describes the design at a higher level, using always blocks and control statements to define how it works. Each approach is used based on the level of detail and abstraction needed.

- **Comparator Design Consideration**

  When designing a comparator, it's important to consider how to handle both signed and unsigned numbers. For unsigned numbers, the comparison is straightforward, as all the bits represent positive values. However, for signed numbers, the most significant bit (MSB) indicates whether the number is positive or negative, making the comparison process more complex. A well-designed comparator must account for these differences and be flexible enough to handle various

bit-widths. It should be capable of performing different types of comparisons, such as checking if one number is greater than, less than, or equal to another.

- **Simulation and Testing in Verilog**

  Simulation is an important step to check if a digital design works correctly before building it in hardware. In Verilog, a testbench is used to provide the inputs to the comparator and check the outputs. It helps to make sure the comparator works as expected by testing different cases. Using simulation tools, designers can see how the comparator behaves through waveforms, helping them spot any mistakes or problems before the design is finalized.

Design Philosophy

- ## Structural Comparator

```
module comparatorStructural(A, B, clk, EQ, GT, LT);
    // Inputs: Two 6-bit numbers (A and B), clock signal
    input [5:0] A, B;
    input clk;
    // Outputs: Equality, Greater, and Less-than indicators
    output EQ, GT, LT;

    // Internal wires for bitwise operations
    wire [5:0] Bnot, Anot;
    wire [5:0] and1out, and2out;
    wire [5:0] norout;
    wire [10:0] and3out;

    // Generate logic for bitwise comparison
    genvar i;
    generate
        for (i = 0; i < 6; i = i + 1) begin : gen_comparator
            not #(4) (Bnot[i], B[i]);          // Bitwise NOT of B
            not #(4) (Anot[i], A[i]);          // Bitwise NOT of A
            and #(9) (and1out[i], Anot[i], B[i]); // A < B for each bit
            and #(9) (and2out[i], A[i], Bnot[i]); // A > B for each bit
            nor #(6) (norout[i], and1out[i], and2out[i]); // A == B for each bit
        end
    endgenerate

    // Generate carry-based comparison for A < B
    and #(9) (and3out[10], norout[5], and1out[4]);
    and #(9) (and3out[9], norout[5], norout[4], and1out[3]);
    and #(9) (and3out[8], norout[5], norout[4], norout[3], and1out[2]);
    and #(9) (and3out[7], norout[5], norout[4], norout[3], norout[2], and1out[1]);
    and #(9) (and3out[6], norout[5], norout[4], norout[3], norout[2], norout[1], and1out[0]);

    or #(9) (LT, and1out[5], and3out[10], and3out[9], and3out[8], and3out[7], and3out[6]); // Output: A < B

    // Generate carry-based comparison for A > B
    and #(9) (and3out[5], norout[5], and2out[4]);
    and #(9) (and3out[4], norout[5], norout[4], and2out[3]);
    and #(9) (and3out[3], norout[5], norout[4], norout[3], and2out[2]);
    and #(9) (and3out[2], norout[5], norout[4], norout[3], norout[2], and2out[1]);
    and #(9) (and3out[1], norout[5], norout[4], norout[3], norout[2], norout[1], and2out[0]);

    or #(9) (GT, and2out[5], and3out[5], and3out[4], and3out[3], and3out[2], and3out[1]); // Output: A > B

    and #(9) (EQ, norout[5], norout[4], norout[3], norout[2], norout[1], norout[0]); // Output: A == B
endmodule

////////////////////////////////////////////////////////////////////////////////

module signedOrUnsignedStructural(A, B, S, clk, EQ, GT, LT);
    // Inputs: Two 6-bit numbers (A and B), Selection bit (S), clock signal
    input [5:0] A, B;
    input S, clk;
    // Outputs: Equality, Greater, and Less-than indicators
    output reg EQ, GT, LT;

    // Internal wires for two's complement and comparison results
    wire [5:0] A_twos_comp, B_twos_comp;
    wire EQ_unsigned, GT_unsigned, LT_unsigned;
    wire EQ_signed, GT_signed, LT_signed;

    // Registers for synchronizing inputs
    reg [5:0] A_reg, B_reg;
    reg S_reg;

    // Capture inputs on the negative edge of the clock
    always @(negedge clk) begin
        A_reg <= A;
        B_reg <= B;
        S_reg <= S;
    end

    // Compute two's complement for signed numbers
    assign A_twos_comp = (A_reg[5] == 1) ? (~A_reg + 1) : A_reg;
    assign B_twos_comp = (B_reg[5] == 1) ? (~B_reg + 1) : B_reg;

    // Instantiate unsigned and signed comparators
    comparatorStructural comparator_unsigned (.A(A_reg), .B(B_reg), .clk(clk), .EQ(EQ_unsigned), .GT(GT_unsigned), .LT(LT_unsigned));
    comparatorStructural comparator_signed (.A(A_twos_comp), .B(B_twos_comp), .clk(clk), .EQ(EQ_signed), .GT(GT_signed), .LT(LT_signed));

    // Select the appropriate comparison result based on S
    always @(posedge clk) begin
        if (S_reg == 0) begin // Unsigned comparison
            EQ <= EQ_unsigned;
            GT <= GT_unsigned;
            LT <= LT_unsigned;
        end else begin // Signed comparison
            EQ <= (A_reg[5] == 0 && B_reg[5] == 1) || (A_reg[5] == 1 && B_reg[5] == 0) ? 0 : EQ_signed; // Handle sign mismatch
            GT <= (A_reg[5] == 0 && B_reg[5] == 1) ? 1 :
                  (A_reg[5] == 1 && B_reg[5] == 0) ? 0 :
                  (A_reg == 6'b100000 && B_reg == 6'b100000) ? 0 :
                  (A_reg[5] == 1 && B_reg[5] == 1) ? ~GT_signed : GT_signed;
            LT <= (A_reg[5] == 0 && B_reg[5] == 1) ? 0 :
                  (A_reg[5] == 1 && B_reg[5] == 0) ? 1 :
                  (A_reg == 6'b100000 && B_reg == 6'b100000) ? 0 :
                  (A_reg[5] == 1 && B_reg[5] == 1) ? ~LT_signed : LT_signed;
        end
    end
endmodule
```

Figure 1: Structural comparator code.

Discussion:

The comparatorStructural module was a combinational logic design that compared two 6-bit numbers, A and B, to determine their relationship: equal (EQ), greater than (GT), or less than (LT). It used a purely structural approach, employing basic gates such as NOT, AND, and NOR. Each bit of A and B was compared individually, and the results were combined to calculate the final outputs.

The signedOrUnsignedStructural module built upon this by adding registers and using a clock signal (clk) to make the circuit synchronous. Registers were used to store the input values (A, B, and S) on the falling edge of the clock. This ensured that the inputs were stable during the comparison process, which was essential in a sequential design. The use of a clock also synchronized the outputs with the rest of the system, avoiding glitches or unstable outputs during transitions.

In this module, two separate comparators were instantiated: one for unsigned comparisons and one for signed comparisons. The unsigned comparator worked directly with the inputs, while the signed comparator transformed the inputs into their 2's complement representation when the numbers were negative (based on their MSB). The correct results were selected based on the S signal: if S was 0, the unsigned comparator's outputs were used; if S was 1, the signed comparator's outputs were selected.

The module also contained logic to handle signed-specific edge cases, such as when one number was positive and the other was negative. For example, if A was positive (MSB = 0) and B was negative (MSB = 1), A was automatically greater than B. If A was negative and B was positive, A was automatically less than B. The outputs (EQ, GT, LT) were updated on the rising edge of the clock, making them stable for synchronous systems. This sequential behavior ensured that the results were available only after the inputs were clocked into the registers, preventing unstable or intermediate outputs. By synchronizing the inputs and outputs with the clock, the design became robust and ready for integration into larger digital systems.

- **Behavioral Comparator**

```
module comparatorBehavioral(A, B, S, clk, EQ, GT, LT);
    input [5:0] A, B;  // 6-bit inputs for A and B
    input S, clk;  // Input signal S for signed/unsigned selection and clk for clock
    output reg EQ, GT, LT;  // Outputs for Equal, Greater Than, and Less Than
    reg [5:0] A_reg, B_reg;  // Registers to store the values of A and B
    reg S_reg;  // Register to store the S signal

    // Always block triggered on the falling edge of the clock (for storing inputs)
    always @(negedge clk) begin
        A_reg <= A;  // Store A in A_reg on falling edge of clk
        B_reg <= B;  // Store B in B_reg on falling edge of clk
        S_reg <= S;  // Store S in S_reg on falling edge of clk
    end

    // Always block triggered on the rising edge of the clock (for comparison logic)
    always @(posedge clk) begin
        EQ <= 0;  // Reset the outputs at the start of every clock cycle
        GT <= 0;
        LT <= 0;

        if (S_reg == 0) begin  // Unsigned comparison when S_reg is 0
            if (A_reg == B_reg)
                EQ <= 1;  // Set EQ if A equals B
            else if (A_reg > B_reg)
                GT <= 1;  // Set GT if A is greater than B
            else
                LT <= 1;  // Set LT if A is less than B
        end
        else begin  // Signed comparison when S_reg is 1
            if (A_reg[5] == 0 && B_reg[5] == 1)  // A is positive, B is negative
                GT <= 1;  // A is greater than B
            else if (A_reg[5] == 1 && B_reg[5] == 0)  // A is negative, B is positive
                LT <= 1;  // A is less than B
            else begin  // Both A and B have the same sign
                if (A_reg[5] == 0 && B_reg[5] == 0) begin  // Both A and B are positive
                    if (A_reg == B_reg)
                        EQ <= 1;  // Set EQ if A equals B
                    else if (A_reg > B_reg)
                        GT <= 1;  // Set GT if A is greater than B
                    else
                        LT <= 1;  // Set LT if A is less than B
                end
                else if (A_reg[5] == 1 && B_reg[5] == 1) begin  // Both A and B are negative
                    if (A_reg == B_reg)
                        EQ <= 1;  // Set EQ if A equals B
                    else if ((~A_reg + 1) < (~B_reg + 1))
                        GT <= 1;  // Set GT if A is less than B (by comparing 2's complement)
                    else
                        LT <= 1;  // Set LT if A is greater than B
                end
            end
        end
    end
endmodule
```

Figure 2: Behavioral comparator code.

Discussion:

The comparatorBehavioral module is a synchronous design used to compare two 6-bit numbers, A and B, and determine whether they are equal (EQ), greater than (GT), or less than (LT). The module uses the clock signal (clk) to synchronize the process, with a register-based approach. On the falling edge of the clock, it stores the input values (A, B, and the signed/unsigned selector S) into registers to ensure stable inputs for comparison. The comparison process occurs on the rising edge of the clock. When the S signal is 0 (unsigned comparison), it directly compares the values of A and B. If S is 1 (signed comparison), the module first checks the most significant bit (MSB) of A and B to determine their sign. If one number is positive and the other is negative, the result is determined immediately: a positive number is greater than a negative number. If both numbers have the same sign, the comparison proceeds by comparing their actual values, with negative numbers being handled using 2's

complement. The module ensures that the outputs are stable and updated only after the inputs are clocked into the registers, preventing any glitches or intermediate states.

- **Testbench**

```
////////////////////////////////////////////////////////////////////////////////
module tb_comparator;
    // Declare 6-bit registers for A and B, and registers for S (signed/unsigned selector) and clk (clock signal)
    reg [5:0] A, B;
    reg S, clk;

    // Declare wires for the outputs of the two comparator modules (structural and behavioral)
    wire EQ_structural, GT_structural, LT_structural;
    wire EQ_behavioral, GT_behavioral, LT_behavioral;

    // Clock generation process: toggles clk every 37 time units to create a clock period of 74 units
    always begin
        #37 clk = ~clk;
    end

    // Instantiate the two comparator modules (behavioral and structural)
    comparatorBehavioral behavioral(A, B, S, clk, EQ_behavioral, GT_behavioral, LT_behavioral);
    signedOrUnsignedStructural structural(A, B, S, clk, EQ_structural, GT_structural, LT_structural);

    initial begin
        // Initialize signals
        clk = 0;  // Set initial clock value
        A = 6'b000000;  // Initialize A to 0
        B = 6'b000000;  // Initialize B to 0
        S = 0;  // Set S to 0 (unsigned comparison)

        // Run the testbench for 8192 cycles
        repeat (8192) begin
            // Increment values of A, B, and S
            {A, B, S} = {A, B, S} + 1;

            // Wait for 74 time units before displaying the results
            #74;

            // Display the current values of A, B, S, and the results of both comparators
            $display("Time:%0t | A:%b B:%b S:%b | Structural [EQ:%b GT:%b LT:%b] | Behavioral [EQ:%b GT:%b LT:%b] | Test Result: %s",
                     $time, A, B, S, EQ_structural, GT_structural, LT_structural, EQ_behavioral, GT_behavioral, LT_behavioral,
                     // Check if outputs from both comparators match and display "PASS" or "FAIL"
                     (EQ_structural == EQ_behavioral && GT_structural == GT_behavioral && LT_structural == LT_behavioral) ? "PASS" : "FAIL");

            // Increment values of A, B, and S again for the next cycle
            {A, B, S} = {A, B, S} + 1;
        end
    end
endmodule

////////////////////////////////////////////////////////////////////////////////
```

Figure 3: Testbench code for all cases.

Discussion:

The testbench is designed to verify the functionality of two comparator modules: the behavioral and structural comparators. It checks whether both modules produce identical outputs—EQ, GT, and LT—for all combinations of 6-bit inputs A and B, as well as the signed/unsigned selector S. The test iterates through all possible combinations of these inputs, and after each cycle, the outputs from both comparators are compared. If the outputs match, the test passes; otherwise, it fails.

The clock period is set to 74 time units, which allows enough time for the comparators to process the inputs. This clock period is based on the maximum latency of the comparator, which is calculated by considering the delays of various gates in the design. Specifically, the latency includes the following gate delays: 4 time units for the NOT gate, 9 time units for the AND gate, 6 time units for the NOR gate, another 9 time units for an AND gate, and 9 time units for an OR gate. The sum of these delays

results in the longest path latency of 37 time units. Consequently, the clock period is set to 74 time units to accommodate this latency, ensuring the correct operation of the comparators without timing violations. The maximum clock frequency is therefore approximately 1/74 time units.

The testbench runs for a total of 8192 cycles, covering all input combinations for A, B, and S. After each cycle, the testbench waits for 74 time units to allow both comparators to compute their outputs. The current values of A, B, S, and the results of both comparators are displayed, and the test outputs "PASS" if the results from both comparators match, or "FAIL" if there is a discrepancy. This comprehensive approach ensures thorough verification of both comparators across all possible input scenarios and under the specified timing conditions.

```
////////////////////////////////////////////////////////////////////////////////
module tb_comparator;
    reg [5:0] A, B;
    reg S, clk;
    wire EQ_structural, GT_structural, LT_structural;
    wire EQ_behavioral, GT_behavioral, LT_behavioral;

    always begin
        #40 clk = ~clk;
    end

    comparatorBehavioral behavioral(A, B, S, clk, EQ_behavioral, GT_behavioral, LT_behavioral);
    signedOrUnsignedStructural structural(A, B, S, clk, EQ_structural, GT_structural, LT_structural);

    initial begin
        clk = 0;

        S = 0; A = 6'b000011; B = 6'b000101; #80;
        $display("Time:%0t | A:%b B:%b S:%b | Structural [EQ:%b GT:%b LT:%b] | Behavioral [EQ:%b GT:%b LT:%b] | Test Result: %s",
                 $time, A, B, S, EQ_structural, GT_structural, LT_structural, EQ_behavioral, GT_behavioral, LT_behavioral,
                 (EQ_structural == EQ_behavioral && GT_structural == GT_behavioral && LT_structural == LT_behavioral) ? "PASS" : "FAIL");

        S = 0; A = 6'b001010; B = 6'b000011; #80;
        $display("Time:%0t | A:%b B:%b S:%b | Structural [EQ:%b GT:%b LT:%b] | Behavioral [EQ:%b GT:%b LT:%b] | Test Result: %s",
                 $time, A, B, S, EQ_structural, GT_structural, LT_structural, EQ_behavioral, GT_behavioral, LT_behavioral,
                 (EQ_structural == EQ_behavioral && GT_structural == GT_behavioral && LT_structural == LT_behavioral) ? "PASS" : "FAIL");

        S = 0; A = 6'b000101; B = 6'b000101; #80;
        $display("Time:%0t | A:%b B:%b S:%b | Structural [EQ:%b GT:%b LT:%b] | Behavioral [EQ:%b GT:%b LT:%b] | Test Result: %s",
                 $time, A, B, S, EQ_structural, GT_structural, LT_structural, EQ_behavioral, GT_behavioral, LT_behavioral,
                 (EQ_structural == EQ_behavioral && GT_structural == GT_behavioral && LT_structural == LT_behavioral) ? "PASS" : "FAIL");

        S = 1; A = 6'b001111; B = 6'b101011; #80;
        $display("Time:%0t | A:%b B:%b S:%b | Structural [EQ:%b GT:%b LT:%b] | Behavioral [EQ:%b GT:%b LT:%b] | Test Result: %s",
                 $time, A, B, S, EQ_structural, GT_structural, LT_structural, EQ_behavioral, GT_behavioral, LT_behavioral,
                 (EQ_structural == EQ_behavioral && GT_structural == GT_behavioral && LT_structural == LT_behavioral) ? "PASS" : "FAIL");

        S = 1; A = 6'b101011; B = 6'b000101; #80;
        $display("Time:%0t | A:%b B:%b S:%b | Structural [EQ:%b GT:%b LT:%b] | Behavioral [EQ:%b GT:%b LT:%b] | Test Result: %s",
                 $time, A, B, S, EQ_structural, GT_structural, LT_structural, EQ_behavioral, GT_behavioral, LT_behavioral,
                 (EQ_structural == EQ_behavioral && GT_structural == GT_behavioral && LT_structural == LT_behavioral) ? "PASS" : "FAIL");

        S = 1; A = 6'b111101; B = 6'b111100; #80;
        $display("Time:%0t | A:%b B:%b S:%b | Structural [EQ:%b GT:%b LT:%b] | Behavioral [EQ:%b GT:%b LT:%b] | Test Result: %s",
                 $time, A, B, S, EQ_structural, GT_structural, LT_structural, EQ_behavioral, GT_behavioral, LT_behavioral,
                 (EQ_structural == EQ_behavioral && GT_structural == GT_behavioral && LT_structural == LT_behavioral) ? "PASS" : "FAIL");

        S = 1; A = 6'b111111; B = 6'b111111; #80;
        $display("Time:%0t | A:%b B:%b S:%b | Structural [EQ:%b GT:%b LT:%b] | Behavioral [EQ:%b GT:%b LT:%b] | Test Result: %s",
                 $time, A, B, S, EQ_structural, GT_structural, LT_structural, EQ_behavioral, GT_behavioral, LT_behavioral,
                 (EQ_structural == EQ_behavioral && GT_structural == GT_behavioral && LT_structural == LT_behavioral) ? "PASS" : "FAIL");

        S = 1; A = 6'b100000; B = 6'b100000; #80;
        $display("Time:%0t | A:%b B:%b S:%b | Structural [EQ:%b GT:%b LT:%b] | Behavioral [EQ:%b GT:%b LT:%b] | Test Result: %s",
                 $time, A, B, S, EQ_structural, GT_structural, LT_structural, EQ_behavioral, GT_behavioral, LT_behavioral,
                 (EQ_structural == EQ_behavioral && GT_structural == GT_behavioral && LT_structural == LT_behavioral) ? "PASS" : "FAIL");


        S = 0; A = 6'b000000; B = 6'b000000; #80;
        $display("Time:%0t | A:%b B:%b S:%b | Structural [EQ:%b GT:%b LT:%b] | Behavioral [EQ:%b GT:%b LT:%b] | Test Result: %s",
                 $time, A, B, S, EQ_structural, GT_structural, LT_structural, EQ_behavioral, GT_behavioral, LT_behavioral,
                 (EQ_structural == EQ_behavioral && GT_structural == GT_behavioral && LT_structural == LT_behavioral) ? "PASS" : "FAIL");

        S = 0; A = 6'b011111; B = 6'b001111; #80;
        $display("Time:%0t | A:%b B:%b S:%b | Structural [EQ:%b GT:%b LT:%b] | Behavioral [EQ:%b GT:%b LT:%b] | Test Result: %s",
                 $time, A, B, S, EQ_structural, GT_structural, LT_structural, EQ_behavioral, GT_behavioral, LT_behavioral,
                 (EQ_structural == EQ_behavioral && GT_structural == GT_behavioral && LT_structural == LT_behavioral) ? "PASS" : "FAIL");
    end
endmodule
////////////////////////////////////////////////////////////////////////////////
```

Figure 4: Testbench code for some cases.

Discussion:

his Verilog testbench code is designed to verify the structural and behavioral comparators with a set of specific test values for A, B, and S. It doesn't test all possible inputs but checks selected cases, displaying the results (EQ, GT, LT) for both comparators and marking the test as "PASS" or "FAIL" based on whether the outputs match.

Simulation Results

```
# KERNEL: Time:605172 | A:111111 B:110001 S:1 | Structural [EQ:0 GT:1 LT:0] | Behavioral [EQ:0 GT:1 LT:0] | Test Result: PASS
# KERNEL: Time:605246 | A:111111 B:110010 S:1 | Structural [EQ:0 GT:1 LT:0] | Behavioral [EQ:0 GT:1 LT:0] | Test Result: PASS
# KERNEL: Time:605320 | A:111111 B:110011 S:1 | Structural [EQ:0 GT:1 LT:0] | Behavioral [EQ:0 GT:1 LT:0] | Test Result: PASS
# KERNEL: Time:605394 | A:111111 B:110100 S:1 | Structural [EQ:0 GT:1 LT:0] | Behavioral [EQ:0 GT:1 LT:0] | Test Result: PASS
# KERNEL: Time:605468 | A:111111 B:110101 S:1 | Structural [EQ:0 GT:1 LT:0] | Behavioral [EQ:0 GT:1 LT:0] | Test Result: PASS
# KERNEL: Time:605542 | A:111111 B:110110 S:1 | Structural [EQ:0 GT:1 LT:0] | Behavioral [EQ:0 GT:1 LT:0] | Test Result: PASS
# KERNEL: Time:605616 | A:111111 B:110111 S:1 | Structural [EQ:0 GT:1 LT:0] | Behavioral [EQ:0 GT:1 LT:0] | Test Result: PASS
# KERNEL: Time:605690 | A:111111 B:111000 S:1 | Structural [EQ:0 GT:1 LT:0] | Behavioral [EQ:0 GT:1 LT:0] | Test Result: PASS
# KERNEL: Time:605764 | A:111111 B:111001 S:1 | Structural [EQ:0 GT:1 LT:0] | Behavioral [EQ:0 GT:1 LT:0] | Test Result: PASS
# KERNEL: Time:605838 | A:111111 B:111010 S:1 | Structural [EQ:0 GT:1 LT:0] | Behavioral [EQ:0 GT:1 LT:0] | Test Result: PASS
# KERNEL: Time:605912 | A:111111 B:111011 S:1 | Structural [EQ:0 GT:1 LT:0] | Behavioral [EQ:0 GT:1 LT:0] | Test Result: PASS
# KERNEL: Time:605986 | A:111111 B:111100 S:1 | Structural [EQ:0 GT:1 LT:0] | Behavioral [EQ:0 GT:1 LT:0] | Test Result: PASS
# KERNEL: Time:606060 | A:111111 B:111101 S:1 | Structural [EQ:0 GT:1 LT:0] | Behavioral [EQ:0 GT:1 LT:0] | Test Result: PASS
# KERNEL: Time:606134 | A:111111 B:111110 S:1 | Structural [EQ:0 GT:1 LT:0] | Behavioral [EQ:0 GT:1 LT:0] | Test Result: PASS
# KERNEL: Time:606208 | A:111111 B:111111 S:1 | Structural [EQ:1 GT:0 LT:0] | Behavioral [EQ:1 GT:0 LT:0] | Test Result: PASS
endsim
```

Figure 5: Some Results for Testbench1.

Discussion:

The output is correct as shown in Figure 5. For the inputs A = 111111 and B = 111110, the result at time 606134 is: KERNEL: Time:655280 | A:111111 B:111110 S:1 | Structural [EQ:0 GT:1 LT:0] | Behavioral [EQ:0 GT:1 LT:0] | Test Result: PASS. This confirms that both the structural and behavioral comparators provide the same result, indicating A is greater than B, and the test passes as expected.

```
run
# KERNEL: Time:80 | A:000011 B:000101 S:0 | Structural [EQ:0 GT:0 LT:1] | Behavioral [EQ:0 GT:0 LT:1] | Test Result: PASS
# KERNEL: Time:160 | A:001010 B:000011 S:0 | Structural [EQ:0 GT:1 LT:0] | Behavioral [EQ:0 GT:1 LT:0] | Test Result: PASS
# KERNEL: Time:240 | A:000101 B:000101 S:0 | Structural [EQ:1 GT:0 LT:0] | Behavioral [EQ:1 GT:0 LT:0] | Test Result: PASS
# KERNEL: Time:320 | A:001111 B:101011 S:1 | Structural [EQ:0 GT:1 LT:0] | Behavioral [EQ:0 GT:1 LT:0] | Test Result: PASS
# KERNEL: Time:400 | A:101011 B:000101 S:1 | Structural [EQ:0 GT:0 LT:1] | Behavioral [EQ:0 GT:0 LT:1] | Test Result: PASS
# KERNEL: Time:480 | A:111101 B:111100 S:1 | Structural [EQ:0 GT:1 LT:0] | Behavioral [EQ:0 GT:1 LT:0] | Test Result: PASS
# KERNEL: Time:560 | A:111111 B:111111 S:1 | Structural [EQ:1 GT:0 LT:0] | Behavioral [EQ:1 GT:0 LT:0] | Test Result: PASS
# KERNEL: Time:640 | A:100000 B:100000 S:1 | Structural [EQ:1 GT:0 LT:0] | Behavioral [EQ:1 GT:0 LT:0] | Test Result: PASS
# KERNEL: Time:720 | A:000000 B:000000 S:0 | Structural [EQ:1 GT:0 LT:0] | Behavioral [EQ:1 GT:0 LT:0] | Test Result: PASS
# KERNEL: Time:800 | A:011111 B:001111 S:0 | Structural [EQ:0 GT:1 LT:0] | Behavioral [EQ:0 GT:1 LT:0] | Test Result: PASS
```

Figure 6: Results for Testbench2.

Discussion:

For some different test values, the output shown in Figure 6 is correct, and the results are consistent as expected.

```
38      and #(9) (and3out[3], norout[5], norout[4], norout[3], and2out[2]);
39      and #(9) (and3out[2], norout[5], norout[4], norout[3], norout[2], and2out[1]);
40      and #(9) (and3out[1], norout[5], norout[4], norout[3], norout[2], norout[1], and2out[0]);
41
42      or #(9) (GT, and2out[5], and3out[5], and3out[4], and3out[3], and3out[2], and3out[1]); // Output: A > B
43
44      or #(9) (EQ, norout[5], norout[4], norout[3], norout[2], norout[1], norout[0]); // Output: A == B
45  endmodule
46
```

Figure 7: Code with error in line 44 use or gate instead of and gate.

```
# KERNEL: Time:654160 | A:111111 B:110000 S:1 | Structural [EQ:1 GT:1 LT:0] | Behavioral [EQ:0 GT:1 LT:0] | Test Result: FAIL
# KERNEL: Time:654240 | A:111111 B:110001 S:1 | Structural [EQ:1 GT:1 LT:0] | Behavioral [EQ:0 GT:1 LT:0] | Test Result: FAIL
# KERNEL: Time:654320 | A:111111 B:110010 S:1 | Structural [EQ:1 GT:1 LT:0] | Behavioral [EQ:0 GT:1 LT:0] | Test Result: FAIL
# KERNEL: Time:654400 | A:111111 B:110011 S:1 | Structural [EQ:1 GT:1 LT:0] | Behavioral [EQ:0 GT:1 LT:0] | Test Result: FAIL
# KERNEL: Time:654480 | A:111111 B:110100 S:1 | Structural [EQ:1 GT:1 LT:0] | Behavioral [EQ:0 GT:1 LT:0] | Test Result: FAIL
# KERNEL: Time:654560 | A:111111 B:110101 S:1 | Structural [EQ:1 GT:1 LT:0] | Behavioral [EQ:0 GT:1 LT:0] | Test Result: FAIL
# KERNEL: Time:654640 | A:111111 B:110110 S:1 | Structural [EQ:1 GT:1 LT:0] | Behavioral [EQ:0 GT:1 LT:0] | Test Result: FAIL
# KERNEL: Time:654720 | A:111111 B:110111 S:1 | Structural [EQ:1 GT:1 LT:0] | Behavioral [EQ:0 GT:1 LT:0] | Test Result: FAIL
# KERNEL: Time:654800 | A:111111 B:111000 S:1 | Structural [EQ:1 GT:1 LT:0] | Behavioral [EQ:0 GT:1 LT:0] | Test Result: FAIL
# KERNEL: Time:654880 | A:111111 B:111001 S:1 | Structural [EQ:1 GT:1 LT:0] | Behavioral [EQ:0 GT:1 LT:0] | Test Result: FAIL
# KERNEL: Time:654960 | A:111111 B:111010 S:1 | Structural [EQ:1 GT:1 LT:0] | Behavioral [EQ:0 GT:1 LT:0] | Test Result: FAIL
# KERNEL: Time:655040 | A:111111 B:111011 S:1 | Structural [EQ:1 GT:1 LT:0] | Behavioral [EQ:0 GT:1 LT:0] | Test Result: FAIL
# KERNEL: Time:655120 | A:111111 B:111100 S:1 | Structural [EQ:1 GT:1 LT:0] | Behavioral [EQ:0 GT:1 LT:0] | Test Result: FAIL
# KERNEL: Time:655200 | A:111111 B:111101 S:1 | Structural [EQ:1 GT:1 LT:0] | Behavioral [EQ:0 GT:1 LT:0] | Test Result: FAIL
# KERNEL: Time:655280 | A:111111 B:111110 S:1 | Structural [EQ:1 GT:1 LT:0] | Behavioral [EQ:0 GT:1 LT:0] | Test Result: FAIL
# KERNEL: Time:655360 | A:111111 B:111111 S:1 | Structural [EQ:1 GT:0 LT:0] | Behavioral [EQ:1 GT:0 LT:0] | Test Result: PASS
```

Figure 8: Wrong output.

Discussion:

As shown in Figure 8, the output is false due to an error I made in the code at line 44, where I incorrectly used an OR gate instead of an AND gate. This mistake caused the structural and behavioral results to mismatch. The structural comparator produced a different result from the behavioral one, leading to a failed test.

Block Diagram:

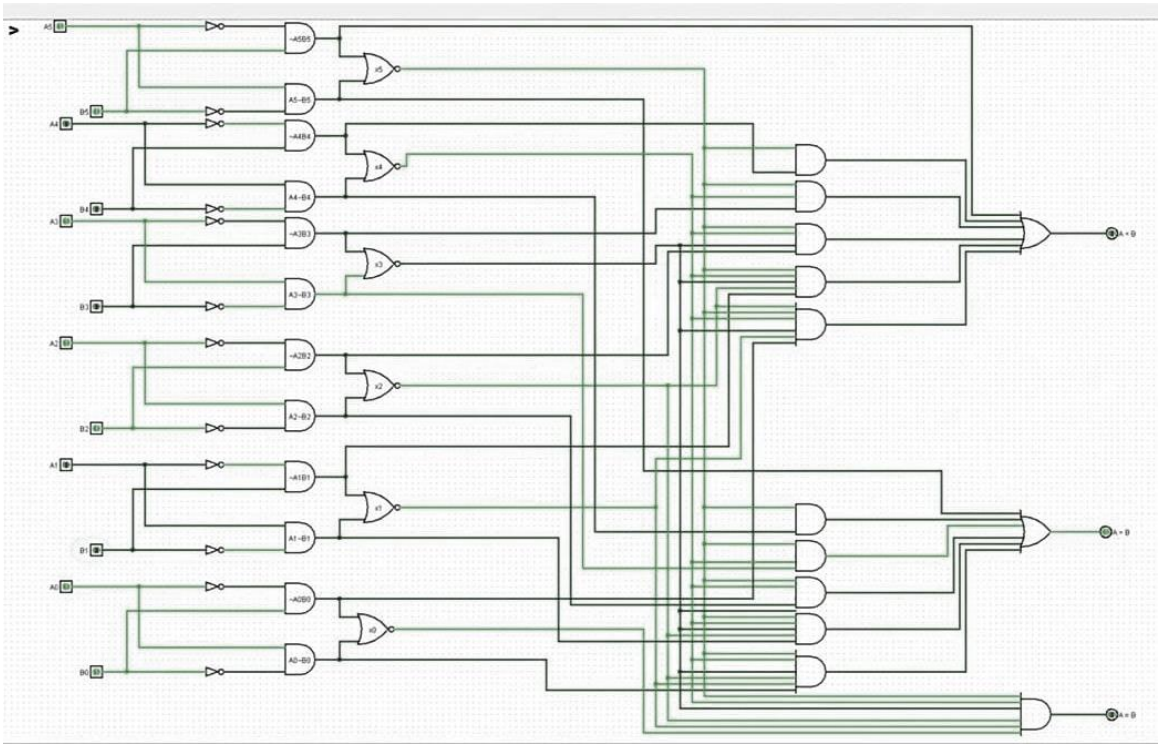

Figure 9:6-bit comparator.

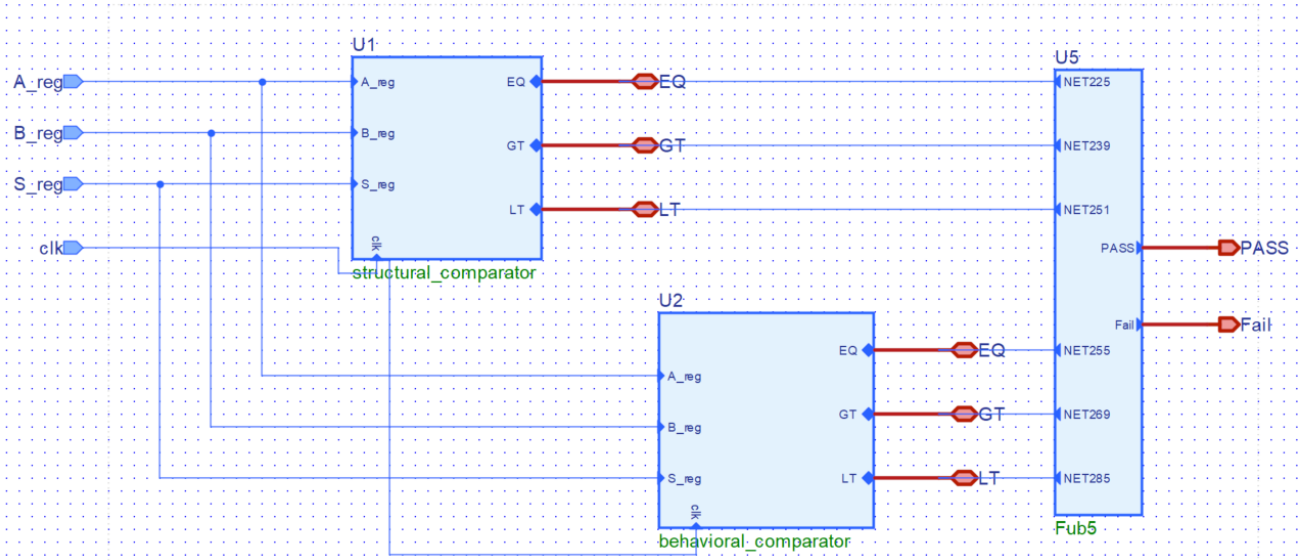

Figure 10: code block diagram.

Conclusion and Future works

In this project, a 6-bit comparator was designed to compare both signed and unsigned numbers using basic logic gates like AND, OR, and NOR. The design was implemented in two ways: structural and behavioral Verilog. The structural design used basic gates and registers for synchronization, while the behavioral design used higher-level Verilog constructs. A testbench was created to check the designs, and the simulation showed that both comparators worked well most of the time. However, an error was found where an AND gate was used instead of an OR gate in the structural design, causing a mismatch and a failed test in some cases. Overall, the project successfully demonstrated how to implement and test comparators in digital systems. The design can also be made more flexible by using parameters instead of a fixed 6-bit size. Additional testing for edge cases and optimizing the design for efficiency are recommended. Automating tests with advanced simulation tools could also improve performance, and implementing the comparator in hardware would provide useful insights into its performance.